System zapisów na zajęcia – USOSv2

Etap 3: projekt wstępny systemu

Kamil Kręcigłowa Bartosz Królik Krzysztof Krówka

28 października 2026

Inżynieria Oprogramowania, zima 2026/2027

Co jest w etapie 3

Zakres (z opisu zadania)

- architektura i stos technologiczny
- przypadki użycia UC0–UC27
- diagramy sekwencji – 7
- diagram ER
- diagramy klas: metody, parametry, wyniki, opis

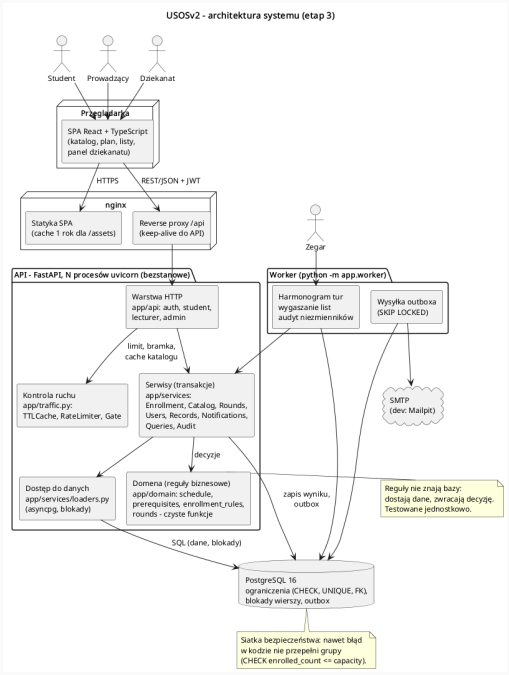
Projekt sprawdzony kodem

- diagramy = działająca implementacja
- 35 testów, test obciążeniowy 20 000 kont

Zmiany względem etapu 2

- **tury per kierunek** – ograniczenie ruchu
- jedna tabela `app_user` z rolą
- **reguły biznesowe w kodzie**, nie w SQL
- kontrola ruchu: cache, limity, 503 + Retry-After
- ślad decyzji dziekanatu (`created_by`)

Architektura


USOSv2 - architektura systemu (etap 3)
Student
Prowadzący
Dziekanat
Przeglądarka
SPA React + TypeScript
(katalog, plan, listy,
panel dziekanatu)
HTTPS
REST/JSON + JWT
nginx
Statyka SPA
(cache 1 rok dla /assets)
Reverse proxy /api
(keep-alive do API)
Zegar
API - FastAPI, N procesów uvicorn (bezstanowe)
Warstwa HTTP
app/api: auth, student,
lecturer, admin
limit, bramka,
cache katalogu
Kontrola ruchu
app/traffic.py:
TTLCache, RateLimiter, Gate
Serwisy (transakcje)
app/services:
Enrollment, Catalog, Rounds,
Users, Records, Notifications,
Queries, Audit
decyzje
Dostęp do danych
app/services/loaders.py
(asyncpg, blokady)
Domena (reguły biznesowe)
app/domain: schedule,
prerequisites, enrollment_rules,
rounds - czyste funkcje
Worker (python -m app.worker)
Harmonogram tur
wygaszanie list
audyt niezmienników
Wysyłka outboxa
(SKIP LOCKED)
SMTP
(dev: Mailpit)
zapis wyniku,
outbox
Reguły nie znają bazy:
dostają dane, zwracają decyzję.
Testowane jednostkowo.
SQL (dane, blokady)
PostgreSQL 16
ograniczenia (CHECK, UNIQUE, FK),
blokady wierszy, outbox
Siatka bezpieczeństwa: nawet błąd
w kodzie nie przepełni grupy
(CHECK enrolled_count <= capacity).

Architektura: zasady

1. **Domena bez bazy.** Reguły (kolizje, wymagania, tury, „jedna grupa typu") to czyste funkcje: dostają dane, zwracają decyzję.
2. **Serwis = transakcja.** Blokady → dane → reguły → zapis → outbox.
3. **Baza = siatka bezpieczeństwa.** `CHECK`, `UNIQUE`, FK odrzucą wynik błędu w kodzie.
4. **Kolejność blokad** student → grupa; awans z listy odwraca ją, więc `NOWAIT` + ponowienie.
5. **Outbox** – powiadomienie w tej samej transakcji, e-mail wysyła worker.
6. **Bezstanowe API** – skaluje się kolejnymi instancjami.

Dlaczego to ważne

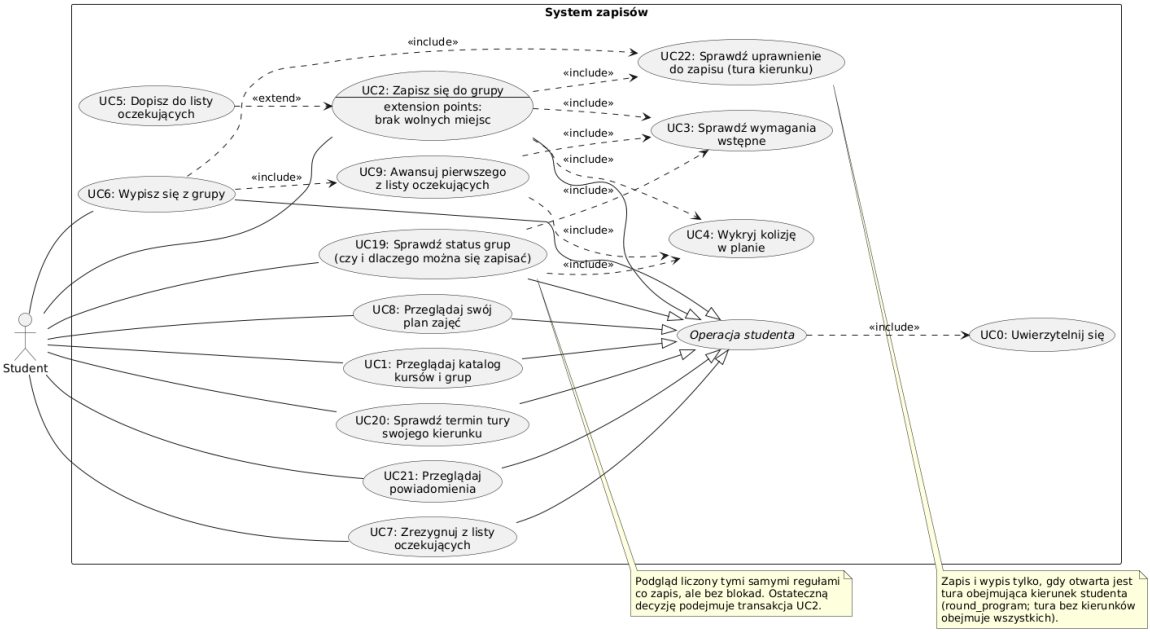
Przy refaktoryzacji błąd zdjął blokadę grupy – `CHECK` odrzucił 11. zapis na 10 miejsc, test go wychwycił.

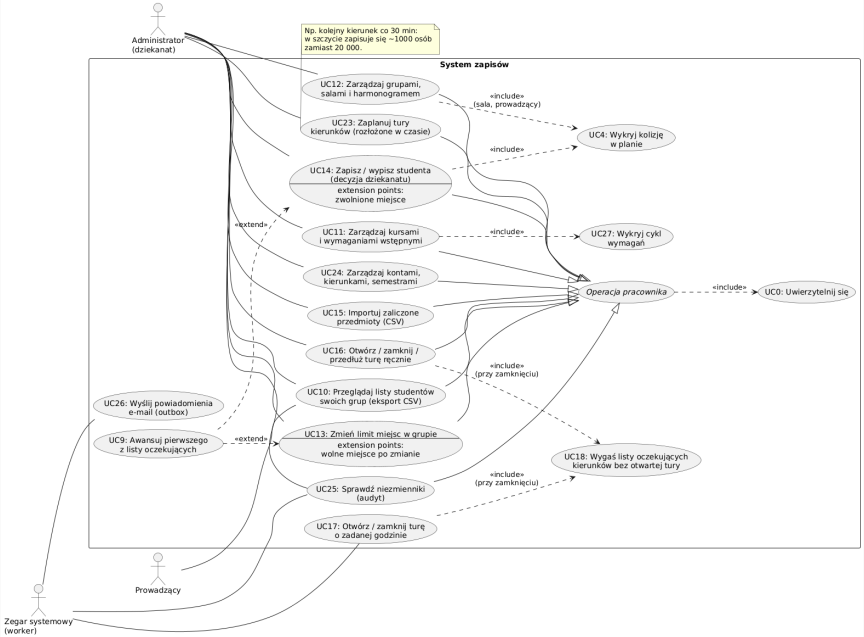
Stos technologiczny

Element	Technologia	Uzasadnienie
Frontend	React 19 + TypeScript, Vite	typowanie, katalog, formularze, siatka planu
Serwer WWW	nginx	statyka z cache – odświeżanie strony nie obciąża API
Backend	Python 3.12, FastAPI, Pydantic	async, walidacja, OpenAPI z kodu
Dostęp do bazy	SQLAlchemy 2 + asyncpg	ORM w panelu; gorąca ścieżka bez ORM (pomiar: ORM = główny koszt CPU)
Baza	PostgreSQL 16	ACID, `FOR UPDATE NOWAIT / SKIP LOCKED`, blokady doradcze, indeksy częściowe
Migracje	Alembic	wersjonowany schemat dla grupy przejmującej
W tle	własny worker (asyncio)	tury i outbox w bazie – broker zbędny
Auth	JWT (12 h), bcrypt	logowanie przed turą, bezstanowe API; docelowo SSO
Testy	pytest na PostgreSQL, k6	blokady i wyścigi tylko na prawdziwej bazie
Wdrożenie	Docker Compose	to samo środowisko dla obu grup

Przypadki użycia

Przypadki użycia (1/2) - student i zapisy (etap 3)




Przypadki użycia (2/2) - dziekanat, prowadzący, zegar (etap 3)
Administrator (dziekanat)
Np. kolejny kierunek co 30 min: w szczycie zapisuje się ~1000 osób zamiast 20 000.
System zapisów
UC12: Zarządzaj grupami, salami i harmonogramem
UC23: Zaplanuj tury kierunków (rozłożone w czasie)
UC14: Zapisz / wypisz studenta (decyzja dziekanatu)
extension points: zwolnione miejsce
UC11: Zarządzaj kursami i wymaganiami wstępnymi
UC24: Zarządzaj kontami, kierunkami, semestrami
UC15: Importuj zaliczone przedmioty (CSV)
UC16: Otwórz / zamknij / przedłuż turę ręcznie
UC10: Przeglądaj listy studentów swoich grup (eksport CSV)
UC13: Zmień limit miejsc w grupie
extension points: wolne miejsce po zmianie
UC25: Sprawdź niezmienniki (audyt)
UC17: Otwórz / zamknij turę o zadanej godzinie
UC4: Wykryj kolizję w planie
UC27: Wykryj cykl wymagań
Operacja pracownika
UC0: Uwierzytelnij się
UC18: Wygaś listy oczekujących kierunków bez otwartej tury
UC26: Wyślij powiadomienia e-mail (outbox)
UC9: Awansuj pierwszego z listy oczekujących
«include» (sala, prowadzący)
«include»
«include»
«include» (przy zamknięciu)
«include» (przy zamknięciu)
«include»
«extend»
«extend»
Prowadzący
Zegar systemowy (worker)

Przypadki użycia: endpointy i błędy (wybór)

UC	Endpoint	Błędy (409 / 429 / 503)
UC2, 5 (student)	`POST /api/me/enrollments`	ROUND_CLOSED, SAME_KIND, PREREQUISITES, CONFLICT, WAITLIST_FULL, RATE_LIMITED, OVERLOADED
UC6, 9 (student)	`DELETE /api/me/enrollments/{s}`	ROUND_CLOSED, NOT_ENROLLED
UC10 (prowadzący)	`GET /api/sections/{s}/roster`	FORBIDDEN (cudza grupa)
UC11 (dziekanat)	`POST /api/admin/courses/{c}/prerequisites`	PREREQUISITE_CYCLE
UC12 (dziekanat)	`POST /api/admin/sections/{s}/meetings`	ROOM_TOO_SMALL, ROOM_BUSY, LECTURER_BUSY
UC13 (dziekanat)	`PATCH /api/admin/sections/{s}`	BELOW_ENROLLED, ROOM_FULL
UC14 (dziekanat)	`POST /api/admin/enrollments`	ALREADY_ENROLLED, CONFLICT, SAME_KIND, ROOM_FULL
UC16, 23 (dziekanat)	`POST /api/admin/rounds, …/{r}/open|close`	BAD_DATES

Pełna tabela UC0–UC27: `etap3/README.md`, sekcja 4.

Uprawnienia ról i obsługa błędów

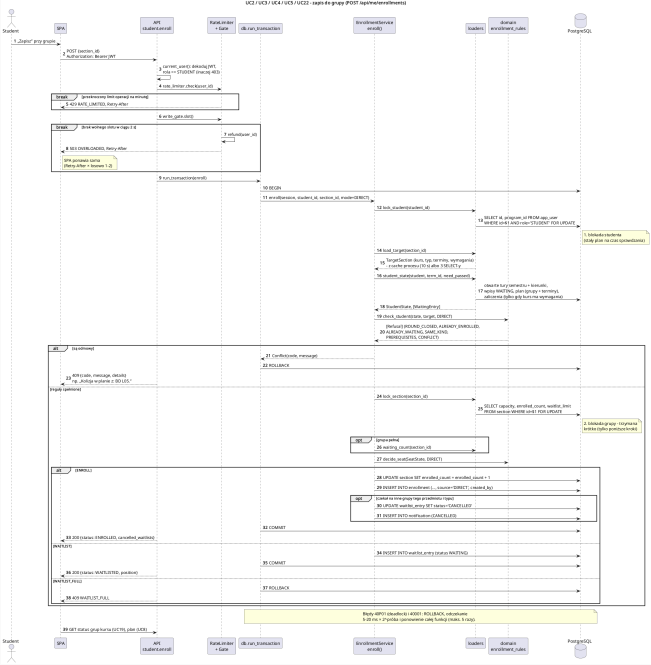
	Stud.	Prow.	Dziek.
katalog	✓	✓	✓
własne zapisy, plan, listy	✓	–	–
listy studentów grupy	–	swoje	✓
katalog, harmonogram, limity	–	–	✓
decyzje o zapisach	–	–	✓
tury, konta, oceny, audyt	–	–	✓

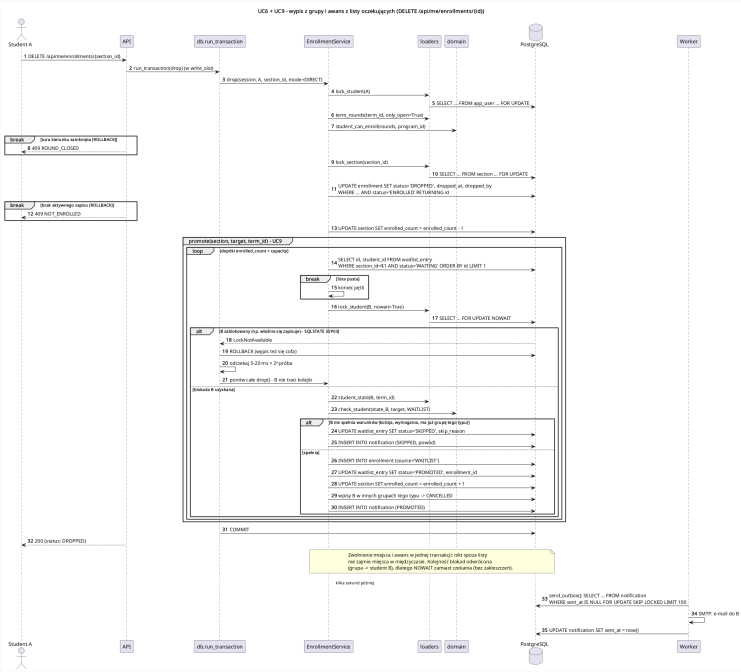
Student nie podaje `student_id` – id pochodzi z tokenu.

Każdy błąd: `{code, message, details}`

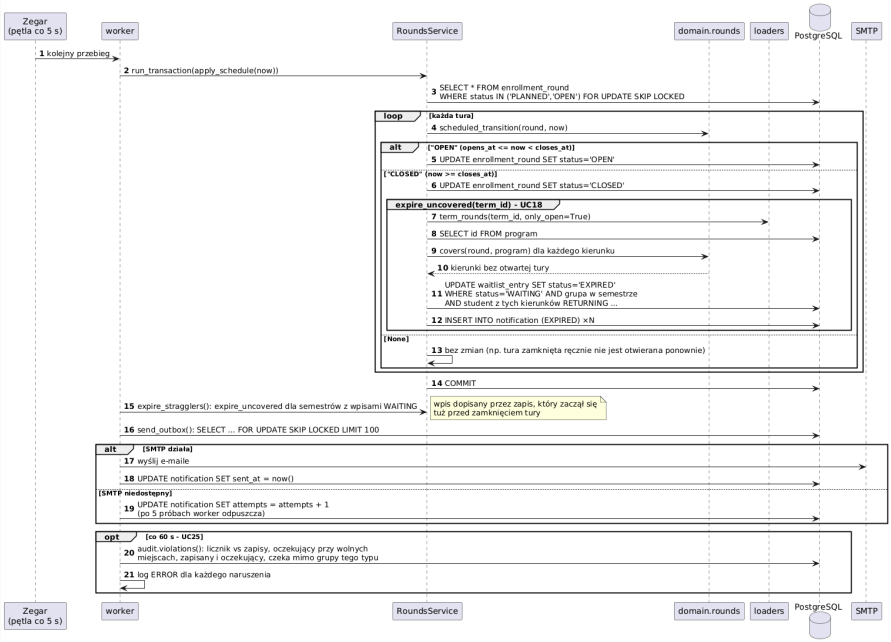
- **409** – odmowa z powodem (*„Kolizja w planie z: BD L05"*)
- **429 / 503** + `Retry-After` – SPA ponawia sama, z rozrzutem
- **401 / 403 / 404 / 422**
- deadlock, `NOWAIT` – ponawiane w transakcji, użytkownik ich nie widzi

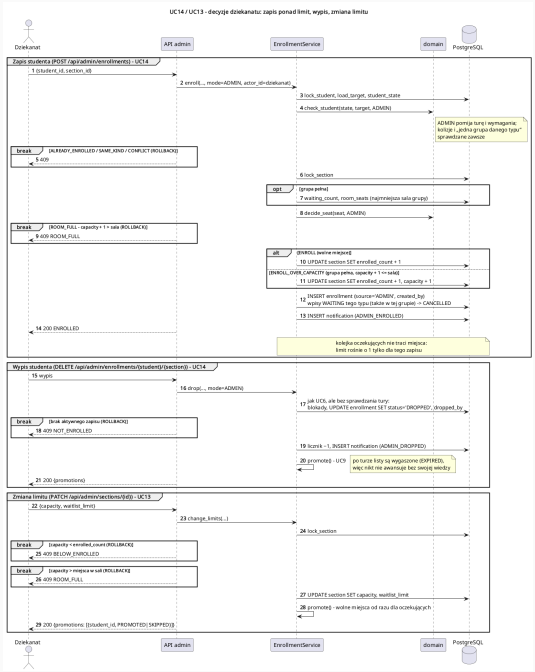
Diagramy sekwencji

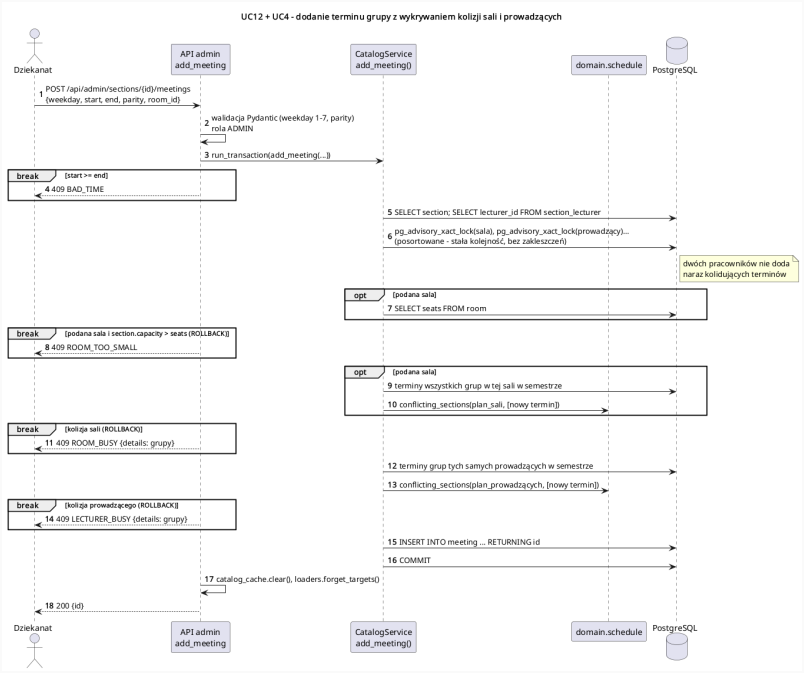

UC2 / UC3 / UC4 / UC5 / UC22 - zapis do grupy (POST /api/me/enrollments)
Student
SPA
API studentcentrali
RateLimiter + Lua
db.run_transaction
EnrollmentService enroll()
loaders
domain enrollment_rules
PostgreSQL
1. „Zapisz" przy grupie
2. POST {section_id} Authorization: Bearer JWT
3. current_user(): dekoduj JWT, weryfikuj podpis i ważność, wyciągnij student_id
4. rate_limiter.check(user_id)
break [przekroczony limit operacji na minutę]
5. 429 RATE_LIMITED, Retry-After
6. err.to_safe_dict()
break [brak wolnego miejsca i tryb FIFO wyłączony]
7. refuse(user_id)
8. 503 OVERLOADED, Retry-After
SPA pokazuje komunikat („Retry-After = liczba sekund")
9. run_transaction(enroll)
10. BEGIN
11. enroll(session, student_id, section_id, mode=DIRECT)
12. lock_student(student_id)
13. SELECT id, program_id FROM app_user WHERE id=$1 AND role='STUDENT' FOR UPDATE
14. load_target(section_id)
15. TargetSection {kurs, typ, termin, wymagania} (cache przez ~5 s, dla 1/30 ECTS)
16. student_state(student, term_id, need_passed)
zlicza ECTS z zaliczonych + bieżących, 17. nazwy WAITING, plan (grupy + terminy), zaliczone (gdzie grupy kursu nie wymagamy)
18. StudentState, [WaitingEntry]
19. check_student(state, target, DIRECT)
(Refusal) FOUND_CLOSED, ALREADY_ENROLLED, PREREQ_NOT_MET, SCHEDULE_CONFLICT, ECTS_LIMIT
alt (sukces)
21. Conflict(code, message)
22 ROLLBACK
409 {code, message, details} np. „Kolizja wydania z ISO 11/5."
23
Pętla ponowień
24 lock_section(section_id)
25 SELECT capacity, enrolled_count, waitlist_limit FROM section WHERE id=$1 FOR UPDATE
2. blokada grupy - inżynieria kolejki (gdzie powstaje kolejka)
opt [przy pełnej]
26 waiting_count(section_id)
27 decide_seat(SeatState, DIRECT)
alt [ENROLL]
28 UPDATE section SET enrolled_count = enrolled_count + 1
29 INSERT INTO enrollment (..., source='DIRECT', created_by)
opt [Udział na innej grupy tego przedmiotu (typu)]
30 UPDATE waitlist_entry SET status='CANCELLED'
31 INSERT INTO waitlist(CANCELLED)
32 COMMIT
33 200 {status: ENROLLED, cancelled_waitlist}
ENROLLED
34 INSERT INTO waitlist_entry (status WAITING)
35 COMMIT
36 200 {status: WAITLISTED, position}
ENROLLED_FULL
37 ROLLBACK
38 409 WAITLIST_FULL
Błędy 409/10 (deadlock) i 40001: ROLLBACK, odczekanie krótkiego czasu i ponowienie (zwykle 1 próba; gdy kolejka jest duża, nawet kilka prób). Po wyczerpaniu limitu prób: 503 SERVICE_BUSY, Retry-After
39 GET /status grupy kursu (GET/9), zdet (UC8)

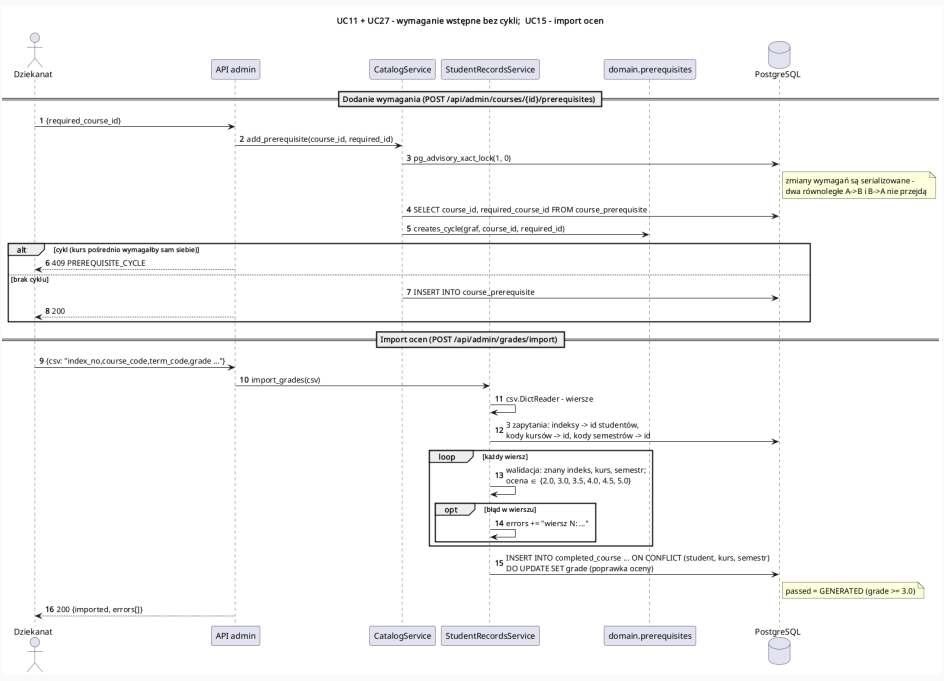

UC6 + UC8 – wypis z grupy i awans z listy oczekujących (DELETE /api/me/enrollments/{id})
Student A
API
db.run_transaction
EnrollmentService
loaders
domain
PostgreSQL
Worker
1 DELETE /api/me/enrollments/{section_id}
2 run_transaction(drop) (w write_slot)
3 drop(session, A, section_id, mode=DIRECT)
4 lock_student(A)
5 SELECT ... FROM app_user ... FOR UPDATE
6 term_rounds(term_id, only_open=True)
7 student_can_enrol(rounds, program_id)
break (Zura klamu wku zarejestrja (ROLLBACK))
8 409 ROUND_CLOSED
9 lock_section(section_id)
10 SELECT ... FROM section ... FOR UPDATE
11 UPDATE enrollment SET status='DROPPED', dropped_at, dropped_by
WHERE ... AND status='ENROLLED' RETURNING id
break (brak aktywnego zapisu (ROLLBACK))
12 403 NOT_ENROLLED
13 UPDATE section SET enrolled_count = enrolled_count - 1
promote(section_target, term_id) - UC9
loop (dopóki enrolled_count < capacity)
14 SELECT id, student_id FROM waitlist_entry
WHERE section_id=$1 AND status='WAITING' ORDER BY id LIMIT 1
break (Brak pozia)
15 koniec pętli
16 lock_student(B, nowait=True)
17 SELECT ... FOR UPDATE NOWAIT
alt (B zablokowany (np. właśnie się zapisuje) - SQLSTATE 55P03)
18 LockNotAvailable
19 ROLLBACK (wypis też się cofa)
20 odczekaj 5-20 ms × 2^próba
21 ponów całe drop() - B nie traci kolejki
[limituje B użytkow]
22 student_state(B, term_id)
23 check_student(state_B, target, WAITLIST)
alt (B nie spełnia warunków (kolizja, wymagania, nie już grupa tego typu))
24 UPDATE waitlist_entry SET status='SKIPPED', skip_reason
25 INSERT INTO notification (SKIPPED, powód)
[spełnia is]
26 INSERT INTO enrollment (source='WAITLIST')
27 UPDATE waitlist_entry SET status='PROMOTED', enrollment_id
28 UPDATE section SET enrolled_count = enrolled_count + 1
29 wypisy B w innych grupach tego typu := CANCELLED
30 INSERT INTO notification (PROMOTED)
31 COMMIT
32 200 (status: DROPPED)
Zwolnienie miejsca i awans w jednej transakcji: nikt spoza listy
nie zajmie miejsca w międzyczasie. Kolejność: blokuj odwrotnie
(grupa → student B), dlatego NOWAIT zamiast czekania (bez zakleszczeń).
kilka sekund później
33 send_outbox() SELECT ... FROM notification
WHERE sent_at IS NULL FOR UPDATE SKIP LOCKED LIMIT 100
34 SMTP: e-mail do B
35 UPDATE notification SET sent_at = now()

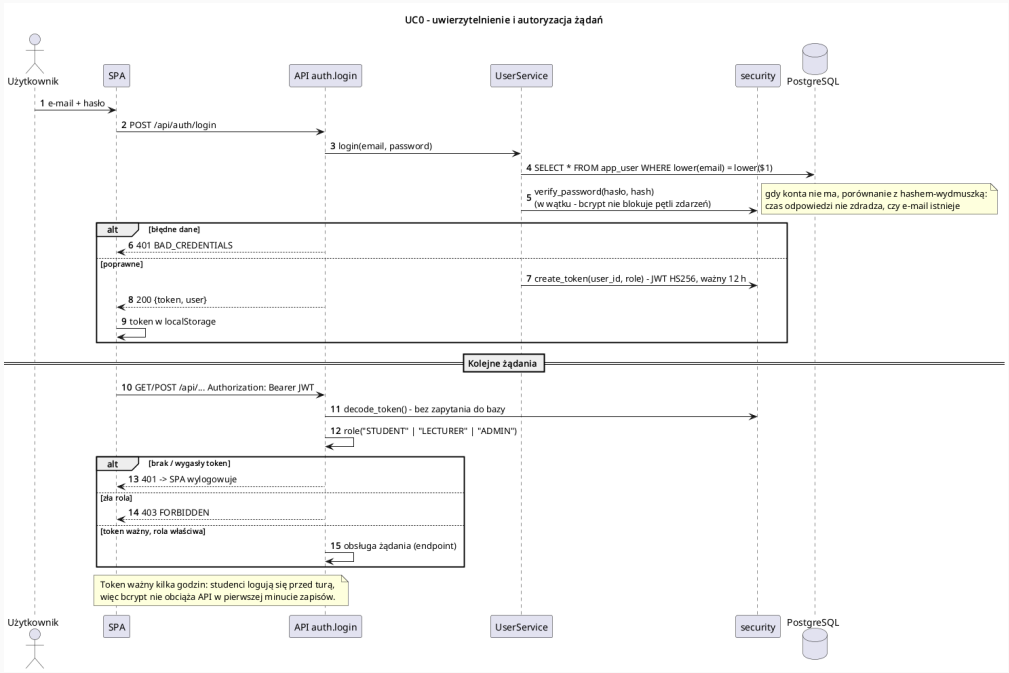
UC17 / UC18 / UC25 / UC26 - worker: tury według harmonogramu, wygaszanie list, outbox, audyt



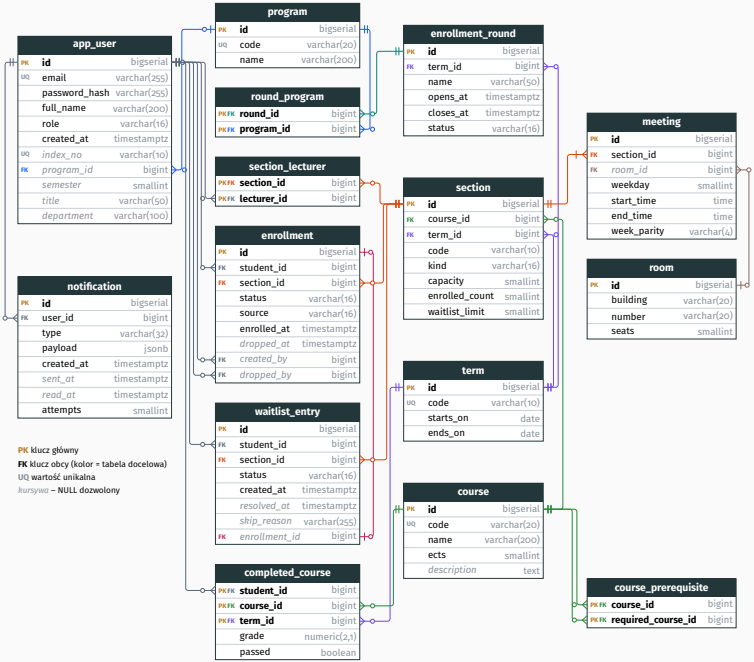

UC14 / UC13 - decyzje dziekanatu: zapis ponad limit, wypis, zmiana limitu
Dziekanat
API admin
EnrollmentService
domain
PostgreSQL
Zapis studenta (POST /api/admin/enrollments) - UC14
1 (student_id, section_id)
2 enroll(..., mode=ADMIN, actor_id=dziekanat)
3 lock_student, load_target, student_state
4 check_student(state, target, ADMIN)
ADMIN pomija tunel i wymagania; kolizje i „pełna grupa dlamego typu" sprawdzane zawsze
break [ALREADY_ENROLLED / SAME_KIND / CONFLICT (ROLLBACK)]
5 409
6 lock_section
opt [grupa pełna]
7 waiting_count, room_seats (najmniejsza sala grupy)
8 decide_seat(seat, ADMIN)
break [ROOM_FULL - capacity + 1 > sala (ROLLBACK)]
9 409 ROOM_FULL
alt [ENROLL (wolne miejsce)]
10 UPDATE section SET enrolled_count + 1
[ENROLL_OVER_CAPACITY (grupa pełna, capacity + 1 <= sala)]
11 UPDATE section SET enrolled_count + 1, capacity + 1
12 INSERT enrollment (source='ADMIN', created_by); wpisy WAITING tego typu (także w tej grupie) -> CANCELLED
13 INSERT notification (ADMIN_ENROLLED)
14 200 ENROLLED
kolejka oczekujących nie traci miejsca: limit rośnie o 1 tylko dla tego zapisu
Wypis studenta (DELETE /api/admin/enrollments/{student}/{section}) - UC14
15 wypis
16 drop(..., mode=ADMIN)
17 jak UC5, ale bez sprawdzania tury; blokady, UPDATE enrollment SET status='DROPPED', dropped_by
break [brak aktywnego zapisu (ROLLBACK)]
18 409 NOT_ENROLLED
19 licznik −1, INSERT notification (ADMIN_DROPPED)
20 promote() - UC9
po turze listy są wygaszone (EXPIRED), więc nikt nie awansuje bez swojej wiedzy
21 200 (promotions)
Zmiana limitu (PATCH /api/admin/sections/{id}) - UC13
22 (capacity, waitlist_limit)
23 change_limits(...)
24 lock_section
break [capacity < enrolled_count (ROLLBACK)]
25 409 BELOW_ENROLLED
break [capacity > miejsca w sali (ROLLBACK)]
26 409 ROOM_FULL
27 UPDATE section SET capacity, waitlist_limit
28 promote() - wolne miejsca od razu dla oczekujących
29 200 (promotions: [{student_id, PROMOTED|SKIPPED}])


UC12 + UC4 - dodanie terminu grupy z wykrywaniem kolizji sali i prowadzących
Dziekanat
API admin add_meeting
CatalogService add_meeting()
domain.schedule
PostgreSQL
1 POST /api/admin/sections/{id}/meetings (weekday, start, end, parity, room_id)
2 walidacja Pydantic (weekday 1-7, parity) rola ADMIN
3 run_transaction(add_meeting(...))
break [start >= end]
4 409 BAD_TIME
5 SELECT section; SELECT lecturer_id FROM section_lecturer
6 pg_advisory_xact_lock(sala), pg_advisory_xact_lock(prowadzący)... (posortowane - stała kolejność, bez zakleszczeń)
dwóch pracowników nie doda naraz kolidujących terminów
opt [podana sala]
7 SELECT seats FROM room
break [podana sala i section.capacity > seats (ROLLBACK)]
8 409 ROOM_TOO_SMALL
opt [podana sala]
9 terminy wszystkich grup w tej sali w semestrze
10 conflicting_sections(plan_sali, [nowy termin])
break [kolizja sali (ROLLBACK)]
11 409 ROOM_BUSY {details: grupy}
12 terminy grup tych samych prowadzących w semestrze
13 conflicting_sections(plan_prowadzących, [nowy termin])
break [kolizja prowadzącego (ROLLBACK)]
14 409 LECTURER_BUSY {details: grupy}
15 INSERT INTO meeting ... RETURNING id
16 COMMIT
17 catalog_cache.clear(), loaders.forget_targets()
18 200 {id}
Dziekanat
API admin add_meeting
CatalogService add_meeting()
domain.schedule
PostgreSQL


UC11 + UC27 - wymaganie wstępne bez cykli; UC15 - import ocen
Dziekanat
API admin
CatalogService
StudentRecordsService
domain.prerequisites
PostgreSQL
Dodanie wymagania (POST /api/admin/courses/{id}/prerequisites)
1 {required_course_id}
2 add_prerequisite(course_id, required_id)
3 pg_advisory_xact_lock(1, 0)
zmiany wymagań są serializowane - dwa równoległe A->B i B->A nie przejdą
4 SELECT course_id, required_course_id FROM course_prerequisite
5 creates_cycle(graf, course_id, required_id)
alt
[cykl (kurs pośrednio wymagałby sam siebie)]
6 409 PREREQUISITE_CYCLE
[brak cyklu]
7 INSERT INTO course_prerequisite
8 200
Import ocen (POST /api/admin/grades/import)
9 {csv: "index_no,course_code,term_code,grade ..."}
10 import_grades(csv)
11 csv.DictReader - wiersze
12 3 zapytania: indeksy -> id studentów, kody kursów -> id, kody semestrów -> id
loop
[każdy wiersz]
13 walidacja: znany indeks, kurs, semestr; ocena ∈ {2.0, 3.0, 3.5, 4.0, 4.5, 5.0}
opt
[błąd w wierszu]
14 errors += "wiersz N: ..."
15 INSERT INTO completed_course ... ON CONFLICT (student, kurs, semestr) DO UPDATE SET grade (poprawka oceny)
passed = GENERATED (grade >= 3.0)
16 200 {imported, errors[]}


UC0 - uwierzytelnienie i autoryzacja żądań
Użytkownik
SPA
API auth.login
UserService
security
PostgreSQL
1 e-mail + hasło
2 POST /api/auth/login
3 login(email, password)
4 SELECT * FROM app_user WHERE lower(email) = lower($1)
5 verify_password(hasło, hash)
(w wątku - bcrypt nie blokuje pętli zdarzeń)
gdy konta nie ma, porównanie z hashem-wydmuszką:
czas odpowiedzi nie zdradza, czy e-mail istnieje
alt
[błędne dane]
6 401 BAD_CREDENTIALS
[poprawne]
7 create_token(user_id, role) - JWT HS256, ważny 12 h
8 200 {token, user}
9 token w localStorage
Kolejne żądania
10 GET/POST /api/... Authorization: Bearer JWT
11 decode_token() - bez zapytania do bazy
12 role("STUDENT" | "LECTURER" | "ADMIN")
alt
[brak / wygasły token]
13 401 -> SPA wylogowuje
[zła rola]
14 403 FORBIDDEN
[token ważny, rola właściwa]
15 obsługa żądania (endpoint)
Token ważny kilka godzin: studenci logują się przed turą,
więc bcrypt nie obciąża API w pierwszej minucie zapisów.
Użytkownik
SPA
API auth.login
UserService
security
PostgreSQL

Model danych


app_user
PK id bigserial
UQ email varchar(255)
password_hash varchar(255)
full_name varchar(200)
role varchar(16)
created_at timestamptz
UQ index_no varchar(10)
FK program_id bigint
semester smallint
title varchar(50)
department varchar(100)
program
PK id bigserial
UQ code varchar(20)
name varchar(200)
round_program
PK FK round_id bigint
PK FK program_id bigint
section_lecturer
PK FK section_id bigint
PK FK lecturer_id bigint
enrollment
PK id bigserial
FK student_id bigint
FK section_id bigint
status varchar(16)
source varchar(16)
enrolled_at timestamptz
dropped_at timestamptz
FK created_by bigint
FK dropped_by bigint
waitlist_entry
PK id bigserial
FK student_id bigint
FK section_id bigint
status varchar(16)
created_at timestamptz
resolved_at timestamptz
skip_reason varchar(255)
FK enrollment_id bigint
completed_course
PK FK student_id bigint
PK FK course_id bigint
PK FK term_id bigint
grade numeric(2,1)
passed boolean
enrollment_round
PK id bigserial
FK term_id bigint
name varchar(50)
opens_at timestamptz
closes_at timestamptz
status varchar(16)
section
PK id bigserial
FK course_id bigint
FK term_id bigint
code varchar(10)
kind varchar(16)
capacity smallint
enrolled_count smallint
waitlist_limit smallint
term
PK id bigserial
UQ code varchar(10)
starts_on date
ends_on date
course
PK id bigserial
UQ code varchar(20)
name varchar(200)
ects smallint
description text
meeting
PK id bigserial
FK section_id bigint
FK room_id bigint
weekday smallint
start_time time
end_time time
week_parity varchar(4)
room
PK id bigserial
building varchar(20)
number varchar(20)
seats smallint
course_prerequisite
PK FK course_id bigint
PK FK required_course_id bigint
notification
PK id bigserial
FK user_id bigint
type varchar(32)
payload jsonb
created_at timestamptz
sent_at timestamptz
read_at timestamptz
attempts smallint
PK klucz główny
FK klucz obcy (kolor = tabela docelowa)
UQ wartość unikalna
kursywa – NULL dozwolony

Model danych: co nowe

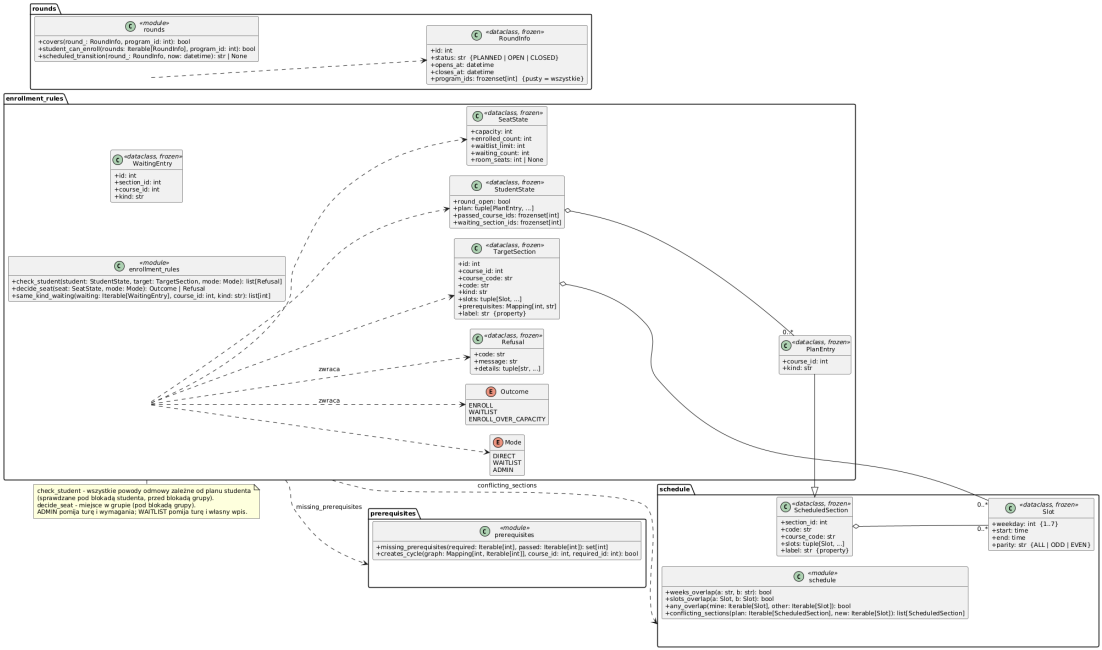
- `program` – kierunek studenta i tur
- `round_program` – tura obejmuje wybrane kierunki; bez wierszy = wszyscy; tury mogą się nakładać
- `app_user` – jedna tabela, dane roli z `CHECK`
- `enrollment.created_by / dropped_by` – kto z dziekanatu
- `notification.read_at, attempts`

Bez zmian z etapu 2: kurs $\neq$ grupa, terminy w `meeting`, licznik `enrolled_count` + `CHECK`, historia zamiast kasowania, kolejność listy = `id`, outbox.

Usunięte z bazy: funkcje kolizji, trigger cykli, widok audytu – przeniesione do kodu.

Projekt klas

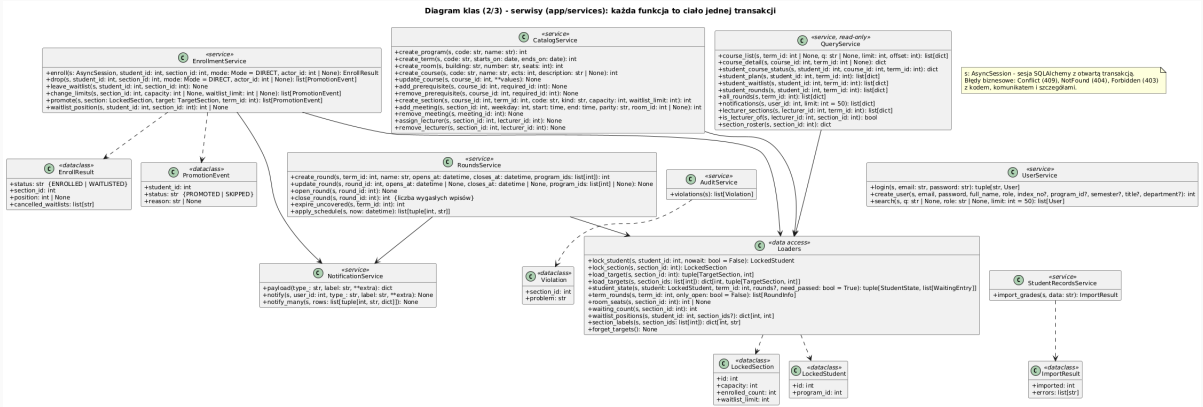
Diagram klas (1/3) - domena: reguły biznesowe (app/domain), bez dostępu do bazy



Domena: kluczowe operacje

Operacja	Zwraca	Funkcjonalność
`check_student(student: StudentState, target: TargetSection, mode: Mode)`	`list[Refusal]`	powody odmowy: tura, już zapisany/czeka, typ grupy, wymagania, kolizje
`decide_seat(seat: SeatState, mode: Mode)`	`Outcome | Refusal`	ENROLL / WAITLIST / ENROLL_OVER_CAPACITY (dziekanat, $\leq$ sala) / odmowa
`conflicting_sections(plan, new: Iterable[Slot])`	`list[Scheduled-Section]`	dzień, godziny (styk to nie kolizja), parzystość tygodni
`creates_cycle(graph, course_id, required_id)`	`bool`	czy nowe wymaganie zamknie cykl
`student_can_enroll(rounds, program_id)`	`bool`	otwarta tura obejmuje kierunek
`scheduled_transition(round, now)`	`"OPEN "CLOSED None`	harmonogram tur dla workera

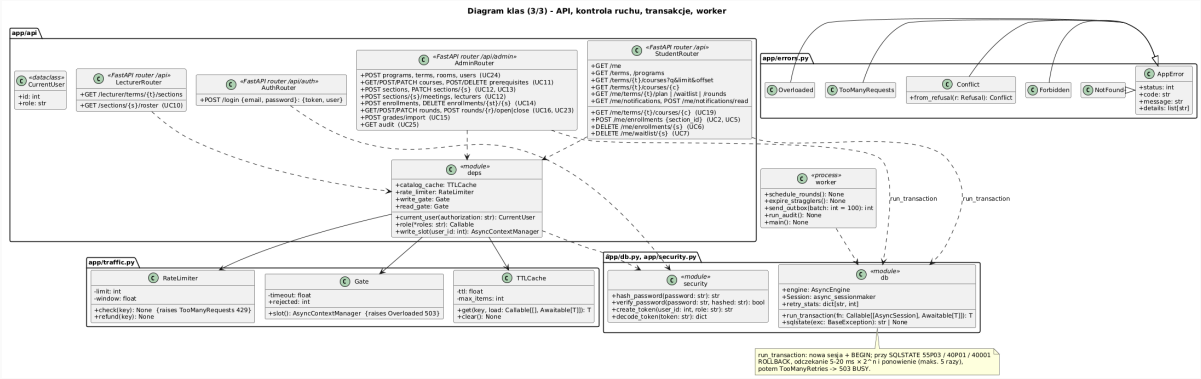
W serwisie: `check_student` (blokada studenta) $\rightarrow$ 409 albo blokada grupy $\rightarrow$ `decide_seat` $\rightarrow$ zapis.


Diagram klas (2/3) - serwisy (app/services): każda funkcja to ciało jednej transakcji
«service»
EnrollmentService
+enrollIs: AsyncSession, student_id: int, section_id: int; mode: Mode = DIRECT, actor_id: int | None): EnrollResult
+dropIs, student_id: int, section_id: int, mode: Mode = DIRECT, actor_id: int | None): list[PromotionEvent]
+leave_waitlistIs, student_id: int, section_id: int): None
+change_limitIs(s, section_id: int, capacity: int | None, waitlist_limit: int | None): list[PromotionEvent]
+promoteIs, section: LockedSection, target: TargetSection, term_id: int): list[PromotionEvent]
+waitlist_positionIs, student_id: int, section_id: int): int | None
«service»
CatalogService
+create_programIs, code: str, name: str): int
+create_termIs, code: str, starts_on: date, ends_on: date): int
+create_roomIs, building: str, number: str, seats: int): int
+create_courseIs, code: str, name: str, ects: int, description: str | None): int
+update_courseIs, course_id: int, **values): None
+add_prerequisiteIs, course_id: int, required_id: int): None
+remove_prerequisiteIs, course_id: int, required_id: int): None
+create_sectionIs, course_id: int, term_id: int, code: str, kind: str, capacity: int, waitlist_limit: int): int
+add_meetingIs, section_id: int, weekday: int, start: time, end: time, parity: str, room_id: int | None): int
+remove_meetingIs, meeting_id: int): None
+assign_lecturerIs, section_id: int, lecturer_id: int): None
+remove_lecturerIs, section_id: int, lecturer_id: int): None
«service, read-only»
QueryService
+course_listIs, term_id: int | None, q: str | None, limit: int, offset: int): list[dict]
+course_detailIs, course_id: int, term_id: int | None): dict
+student_course_statusIs, student_id: int, course_id: int, term_id: int): dict
+student_planIs, student_id: int, term_id: int): list[dict]
+student_waitlistIs, student_id: int, term_id: int): list[dict]
+student_roundsIs, student_id: int, term_id: int): list[dict]
+all_roundsIs, term_id: int): list[dict]
+notificationsIs, user_id: int, limit: int = 50): list[dict]
+lecturer_sectionsIs, lecturer_id: int, term_id: int): list[dict]
+is_lecturer_ofIs, lecturer_id: int, section_id: int): bool
+section_rosterIs, section_id: int): dict
s: AsyncSession - sesja SQLAlchemy z otwartą transakcją.
Błędy biznesowe: Conflict (409), NotFound (404), Forbidden (403)
z kodem, komunikatem i szczegółami.
«dataclass»
EnrollResult
+status: str (ENROLLED | WAITLISTED)
+section_id: int
+position: int | None
+cancelled_waitlists: list[str]
«dataclass»
PromotionEvent
+student_id: int
+status: str (PROMOTED | SKIPPED)
+reason: str | None
«service»
RoundsService
+create_roundIs, term_id: int, name: str, opens_at: datetime, closes_at: datetime, program_ids: list[int]): int
+update_roundIs, round_id: int, opens_at: datetime | None, closes_at: datetime | None, program_ids: list[int] | None): None
+open_roundIs, round_id: int): None
+close_roundIs, round_id: int): int {liczba wygasłych wpisów}
+expire_uncovereIs, term_id: int): int
+apply_schedulels, now: datetime): list[tuple[int, str]]
«service»
AuditService
+violationsIs): list[Violation]
«service»
UserService
+loginIs, email: str, password: str): tuple[str, User]
+create_userIs, email, password, full_name, role, index_no?, program_id?, semester?, title?, department?): int
+searchIs, q: str | None, role: str | None, limit: int = 50): list[User]
«service»
NotificationService
+payloadtype_: str, label: str, **extra): dict
+notifyIs, user_id: int, type_: str, label: str, **extra): None
+notify_manyIs, rows: list[tuple[int, str, dict]]: None
«dataclass»
Violation
+section_id: int
+problem: str
«data access»
Loaders
+lock_studentIs, student_id: int, nowait: bool = False): LockedStudent
+lock_sectionIs, section_id: int): LockedSection
+load_targetIs, section_id: int): tuple[TargetSection, int]
+load_targetsIs, section_ids: list[int]): dict[int, tuple[TargetSection, int]]
+student_statels, student: LockedStudent, term_id: int, rounds?, need_passed: bool = True): tuple[StudentState, list[WaitingEntry]]
+term_roundsIs, term_id: int, only_open: bool = False): list[RoundInfo]
+room_seatsIs, section_id: int): int | None
+waiting_countIs, section_id: int): int
+waitlist_positionsIs, student_id: int, section_ids?): dict[int, int]
+section_labelsIs, section_ids: list[int]): dict[int, str]
+forget_targets(): None
«service»
StudentRecordsService
+import_gradesIs, data: str): ImportResult
«dataclass»
LockedSection
+id: int
+capacity: int
+enrolled_count: int
+waitlist_limit: int
«dataclass»
LockedStudent
+id: int
+program_id: int
«dataclass»
ImportResult
+imported: int
+errors: list[str]

EnrollmentService: operacje

Operacja (`s: AsyncSession` pominięte)	**Zwraca**	**Funkcjonalność**
`enroll(student_id, section_id, mode=DIRECT, actor_id=None)`	`EnrollResult`	UC2/5/14: blokady, reguły, zapis albo lista, anulowanie list tego typu
`drop(student_id, section_id, mode=DIRECT, actor_id=None)`	`list[PromotionEvent]`	UC6/14: wypis (student tylko w turze), potem awans
`leave_waitlist(student_id, section_id)`	`None`	UC7
`change_limits(section_id, capacity=None, waitlist_limit=None)`	`list[PromotionEvent]`	UC13: zapisani $\leq$ limit $\leq$ sala, potem awans
`promote(section, target, term_id)`	`list[PromotionEvent]`	UC9: dopóki są miejsca – `NOWAIT`, ponowne reguły, PROMOTED / SKIPPED
`waitlist_position(student_id, section_id)`	`int | None`	pozycja na liście

Pozostałe serwisy (Catalog, Rounds, Users, Records, Notification, Audit, Query) i `Loaders` – tabele z opisem każdej metody w `etap3/README.md`, sekcja 7.


Diagram klas (3/3) - API, kontrola ruchu, transakcje, worker
app/api
«dataclass»
CurrentUser
+id: int
+role: str
«FastAPI router /api»
LecturerRouter
+GET /lecturer/terms/{t}/sections
+GET /sections/{s}/roster (UC10)
«FastAPI router /api/auth»
AuthRouter
+POST /login {email, password} → {token, user}
«FastAPI router /api/admin»
AdminRouter
+POST programs, terms, rooms, users (UC24)
+GET/POST/PATCH courses, POST/DELETE prerequisites (UC11)
+POST sections, PATCH sections/{s} (UC12, UC13)
+POST sections/{s}/meetings, lecturers (UC12)
+POST enrollments, DELETE enrollments/{st}/{s} (UC14)
+GET/POST/PATCH rounds, POST rounds/{r}/open|close (UC16, UC23)
+POST grades/import (UC15)
+GET audit (UC25)
«FastAPI router /api»
StudentRouter
+GET /me
+GET /terms, /programs
+GET /terms/{t}/courses?q&limit&offset
+GET /terms/{t}/courses/{c}
+GET /me/terms/{t}/plan | /waitlist | /rounds
+GET /me/notifications, POST /me/notifications/read
+GET /me/terms/{t}/courses/{c} (UC19)
+POST /me/enrollments {section_id} (UC2, UC5)
+DELETE /me/enrollments/{s} (UC6)
+DELETE /me/waitlist/{s} (UC7)
«module»
deps
+catalog_cache: TTLCache
+rate_limiter: RateLimiter
+write_gate: Gate
+read_gate: Gate
+current_user(authorization: str): CurrentUser
+role(*roles: str): Callable
+write_slot(user_id: int): AsyncContextManager
app/errors.py
Overloaded
TooManyRequests
Conflict
+from_refusal(r: Refusal): Conflict
Forbidden
NotFound
AppError
+status: int
+code: str
+message: str
+details: list[str]
«process»
worker
+schedule_rounds(): None
+expire_stragglers(): None
+send_outbox(batch: int = 100): int
+run_audit(): None
+main(): None
run_transaction
run_transaction
app/traffic.py
RateLimiter
-limit: int
-window: float
+check(key): None (raises TooManyRequests 429)
+refund(key): None
Gate
-timeout: float
+rejected: int
+slot(): AsyncContextManager (raises Overloaded 503)
TTLCache
+ttl: float
+max_items: int
+get(key, load: Callable[[], Awaitable[T]]): T
+clear(): None
app/db.py, app/security.py
«module»
security
+hash_password(password: str): str
+verify_password(password: str, hashed: str): bool
+create_token(user_id: int, role: str): str
+decode_token(token: str): dict
«module»
db
+engine: AsyncEngine
+Session: async_sessionmaker
+retry_stats: dict[str, int]
+run_transaction(fn: Callable[[AsyncSession], Awaitable[T]]): T
+sqlstate(exc: BaseException): str | None
run_transaction: nowa sesja + BEGIN; przy SQLSTATE 55P03 / 40P01 / 40001 ROLLBACK, odczekanie 5-20 ms × 2^n i ponowienie (maks. 5 razy), potem TooManyRetries -> 503 BUSY.

Duży ruch

20 000 studentów: ograniczyć ruch, potem go obsłużyć

1. Tury per kierunek (UC23)
kolejny kierunek co 30 min $\Rightarrow$ w szczycie ok. 1000 osób zamiast 20 000.

2. Krótkie blokady
reguły planu studenta przed blokadą grupy; pod blokadą grupy tylko kilka zapytań.

3. Kontrola wejścia
limit operacji na studenta (429), bramka równoczesnych transakcji (503 + `Retry-After`), SPA ponawia z rozrzutem.

4. Cache odczytów
katalog 3 s (równoczesne chybienia = jedno zapytanie), dane grupy przy zapisie 10 s.

5. Gorąca ścieżka bez ORM
profilowanie: budowanie zapytań w SQLAlchemy zjadało CPU $\Rightarrow$ asyncpg.

6. Skalowanie
API bezstanowe, statyka z nginx, logowanie przed turą (JWT 12 h).

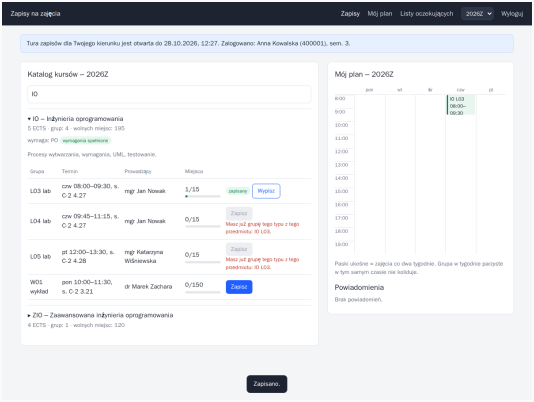
Test obciążeniowy: otwarcie tury (k6)

Każdy student: katalog, status grup, zapis na 6 przedmiotów (wykład + laboratorium), plan. Seed: 20 kierunków, 20 000 kont, 9000 grup. Jedna maszyna (16 rdzeni) razem z k6.

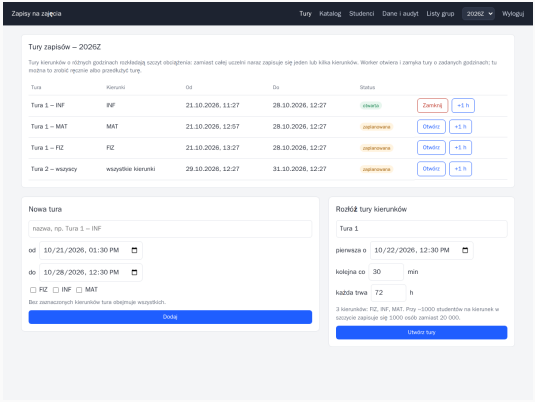
Scenariusz	p95 zapisu	sesja studenta	ponowienia 429/503	niezmienniki
1 kierunek, 1000 osób – pierwsza wersja	2,5 s	2 min	22 873 (1576 nieudanych)	OK
1 kierunek, 1000 osób – po poprawkach	1,9 s	17 s	0	OK
3 kierunki, 3000 osób (przeciążenie)	2,1 s	49 s	18 034 (0,5 % nieudanych)	OK, 0 przepełnień

- wąskie gardło: CPU procesów API (baza ok. 1/4 mocy) ⇒ kolejne instancje API
- pierwsza wersja: limit liczył też ponowienia ⇒ kaskada 429 – znalezione dopiero pomiarem
- przeciążenie: system zwalnia, ale **nie traci spójności** ⇒ tury per kierunek są konieczne

Implementacja: ekrany



student: status grup z powodem odmowy, plan, tura kierunku



dziekanat: tury kierunków, „Rozłóż tury kierunków”

Podsumowanie

Weryfikacja i przekazanie projektu

Weryfikacja

- 35 testów na PostgreSQL: reguły, wyścigi (30 osób / 10 miejsc, dwie karty, awans vs zapis – ścieżka `NOWAIT` faktycznie wykonywana), API, worker
- test obciążeniowy + audyt niezmienników po teście

Dla grupy przejmującej

- `docker compose up` + seed demo/uczelnia
- nowa reguła → `app/domain` + test
- zmiana schematu → migracja Alembic
- znane ograniczenia: kursy nieprzypisane do kierunków, brak zamiany grupy, limit per proces, jedna rola na konto

Dziękujemy

Pytania?